



VINUNIVERSITY

Lab 09: Introduction to graphs and basic graph algorithms.

Week 12, May 11 2026

**College of Engineering and Computer Science,
Object-Oriented Programming & Data Structures Spring**

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your code and functions/modules should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables and functions should have meaningful names, and code should be organized into functions/methods where appropriate.
- Academic honesty is required in all work you submit to be graded. You should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions (modified to provide your student ID, your name, etc).
- You should **upload your .java file(s) to Codeforces before the end of the laboratory session** unless the instructor gave a specified deadline. **No Canvas or Codeforces submission** will get **0** point.
- Submit a separate .java file for each lab problem, with the exact filename specified in each lab question.
- Late submission of lab practice without an approved extension will incur the following penalties:
 1. No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 2. The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 3. The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.
- **Navigate to the Lab:** Open the specific link provided for this week's lab: <https://vinuni26oop.contest.codeforces.com/>.

Prelab Problem 1: Build Adjacency List

Write a Java program that converts an undirected graph from edge list representation into adjacency list representation. An edge list stores graph edges as pairs of vertices, while an adjacency list stores, for each vertex, the list of its neighboring vertices.

Given the vertices and edges of an undirected graph, build and print its adjacency list.

Input

The first line contains two integers: N (number of vertices) and M (number of edges).

The next M lines each contain two integers u and v , representing an undirected edge between vertices.

Output

Print the adjacency list for all vertices from 0 to $N - 1$. For each vertex, print the vertex number followed by a colon and its neighboring vertices in insertion order.

Notes

Vertices are numbered from 0 to $N - 1$.

Examples

Test Case 1:

Input:

```
5 4
0 1
0 2
1 3
2 4
```

Output:

```
0: 1 2
1: 0 3
2: 0 4
3: 1
4: 2
```

Test Case 2:

Input:

```
4 2
0 1
2 3
```

Output:

```
0: 1
1: 0
2: 3
3: 2
```

Problem 2: Degree of Each Vertex

Write a Java program that computes the degree information of each vertex in a graph. In an undirected graph, the degree of a vertex is the number of connected edges. In a directed graph, each vertex has an in-degree (incoming edges) and an out-degree (outgoing edges).

Given the graph type and a list of edges, calculate and print the degree information for all vertices.

Input

The first line contains two integers: N (number of vertices) and M (number of edges).

The second line contains a character representing the graph type:

- U for undirected graph
- D for directed graph

The next M lines each contain two integers u and v , representing an edge.

Output

- For an undirected graph, print the degree of each vertex.
- For a directed graph, print the in-degree and out-degree of each vertex.

Notes

Vertices are numbered from 0 to $N - 1$.

Examples

Test Case 1:

Input:

4 3

U

0 1

1 2

1 3

Output:

0: 1

1: 3

2: 1

3: 1

Test Case 2:

Input:

4 4

D

0 1

2 1

```
1 3  
3 1
```

Output:

```
0: 0 1  
1: 3 1  
2: 0 1  
3: 1 1
```

Problem 3: Graph Property Detection

Write a Java program that determines several properties of a graph. A simple graph has no self-loops and no duplicate edges. A graph may also be directed or undirected, and weighted or unweighted.

Given the graph type and a list of edges, determine whether the graph is simple, whether it is directed or undirected, and whether it is weighted or unweighted.

Input

The first line contains two integers: N (number of vertices) and M (number of edges).

The second line contains two strings:

- D or U for directed or undirected graph
- W or UW for weighted or unweighted graph

The next M lines describe the edges:

- For unweighted graphs: $u\ v$
- For weighted graphs: $u\ v\ w$

Output

Print three lines:

- Simple Graph: Yes or No
- Type: Directed or Undirected
- Weight: Weighted or Unweighted

Notes

A self-loop is an edge from a vertex to itself.

Duplicate edges are considered the same edge repeated multiple times.

Examples

Test Case 1:

Input:

4 3

U UW

0 1

1 2

2 3

Output:

Simple Graph: Yes

Type: Undirected
Weight: Unweighted

Test Case 2:

Input:

3 4

D W

0 1 5

1 1 2

0 1 7

2 0 4

Output:

Simple Graph: No

Type: Directed

Weight: Weighted

Problem 4: BFS Traversal Order

Write a Java program that performs a Breadth First Search (BFS) traversal on an undirected graph starting from a given source vertex. BFS visits vertices level by level using a queue. Starting from the source vertex, visit all directly connected neighbors first before moving to the next level of vertices.

The graph is represented using an adjacency matrix.

Input

The first line contains two integers: N (number of vertices) and M (number of edges).

The next M lines each contain two integers u and v , representing an undirected edge between vertices.

The last line contains the starting vertex s .

Output

Print the BFS traversal order starting from vertex s .

Notes

Vertices are numbered from 0 to $N - 1$.

Visit neighboring vertices in increasing numerical order.

Examples

Test Case 1:

Input:

```
5 4
0 1
0 2
1 3
2 4
0
```

Output:

```
0 1 2 3 4
```

Test Case 2:

Input:

```
4 2
0 1
2 3
2
```

Output:

```
2 3
```

Problem 5: DFS Traversal Order

Write a Java program that performs a Depth First Search (DFS) traversal on an undirected graph starting from a given source vertex. DFS explores as far as possible along one path before backtracking. Starting from the source vertex, visit connected vertices recursively using Depth First Search traversal.

The graph is represented using an adjacency matrix.

Input

The first line contains two integers: N (number of vertices) and M (number of edges).

The next M lines each contain two integers u and v , representing an undirected edge between vertices.

The last line contains the starting vertex s .

Output

Print the DFS traversal order starting from vertex s .

Notes

Vertices are numbered from 0 to $N - 1$.

Visit neighboring vertices in increasing numerical order.

Examples

Test Case 1:

Input:

```
5 4
0 1
0 2
1 3
2 4
0
```

Output:

```
0 1 3 2 4
```

Test Case 2:

Input:

```
4 2
0 1
2 3
2
```

Output:

```
2 3
```